

C08 — Improving an Existing Product

1. What interviewers test

"How would you improve X?" is one of the most common product questions, and it is a different question from "design a product." The product already exists. It has users who rely on it, a business behind it, numbers that move, and a history that explains why it looks the way it does. The interviewer is watching whether you treat it as something that exists or as a blank page.

Where it shows up. Inferred from the round structures candidates commonly report: improvement prompts about a well-known product (a maps app, a messaging app, a notes app, a video service) appear in product-sense rounds at Google, Meta, Microsoft, Amazon, Apple and many others, often as the second or third question type after a pure design prompt [unverified: candidate-reported]. On the job it is the work most associate and early-career PMs do most of the time: a team owns an area of a live product and has to decide what to change.

What is scored. The same five things as a design round (structure, user choice, problem choice, solution quality, a decision with a measure), plus three that belong to this question Inferred: whether you understood what the product is for before changing it, whether you chose where to improve with a reason and some sense of size, and whether you thought about the people the change might hurt.

Six observable behaviors:

1. You state what the product is for, for users and for the company, and what you would look at to understand its current state.
2. You choose an area to improve (a goal, a segment, a step in the journey) by a stated criterion, and say how much room there is.
3. You name a specific problem at that step, with the evidence you have or would collect.
4. You consider more than one kind of move (fix, add, remove, reposition) and choose with a rough size of the gain.
5. You name who loses from the change and what you do about it.
6. You define success with a guardrail and a counter metric, and say what would make you reverse.

The failure this prevents. Six patterns end improvement answers: a wish list for an app the candidate likes, ignoring the existing users as if the product were new, not knowing what the product is for, reaching for a new feature when a fix or a removal would do, no sense of how big the gain could be, and no mention of who is worse off.

2. The concept from first principles

C08.1 How improving differs from designing from scratch

C07 gave a spine for designing for a user. Improving an existing product keeps most of it and changes six things.

In a design round	In an improvement round	Consequence for your answer
No users yet; you pick one	Many users, already using it	Choose which existing users and what they do today
The goal is yours to set	The product has a role and a goal	Infer the goal first; do not invent one that ignores it
No numbers	A funnel, metrics, a history	Say what you would look at, and reason with plausible numbers
Any solution is open	Constraints: habits, data, integrations, other devices	Name the constraint that matters before proposing
A new thing cannot hurt anyone yet	A change can hurt people who rely on the old way	Say who loses and how you protect them
The solution is mostly an addition	Fix, remove and reposition are live options	Offer different kinds of move, not only new features

The key shift is that you are looking for the most valuable change to something that works for many people, and the downside is real. A new feature that helps one segment but breaks a habit for a larger one is a worse product, however clever.

When not to use the improvement spine. If the prompt asks for something new (a product that does not exist), use C07. If the prompt is about entering a market where the product does not yet operate, the questions change again (C09). If a metric has already moved and the question is why, you are diagnosing, not improving (C13). If the interviewer asks for a critique rather than an improvement, the method is in C11.

Counterexample. Asked "How would you improve a maps app?", a candidate says at once, "I'd add augmented-reality walking directions, because it's cool and nobody does it well." The interviewer asks who uses the product and what they do most. The candidate has no answer: they never said what the product is for, which users matter, or where it falls short. The idea might be sound, but there is nothing to show it is the best use of the team's time. A candidate who first says "this is mostly used for driving directions and finding places, so I'd look at which journey step loses the most people" has not yet proposed anything and is already ahead.

Hidden risks. Two. First, claiming knowledge you do not have: "I know their retention is low" is a claim about a company's internal data. Say "I'd expect" and "I'd check," and use public information, labeled as such. Second, anchoring on what you personally dislike. Your own pain is one data point; it is a hypothesis about a segment, not evidence.

C08.2 Reading a product's current state

Before you choose what to improve, form a view of the product as it is. FW-C08-02 gives five questions: what role the product plays, who uses it and how, where the funnel leaks, which metrics are healthy and which are not, and what cannot change. In an interview you will not have the data, so you reason from public information and your own use, say so, and name what you would verify.

Role in the company. A product can be a revenue engine (it earns directly), a growth engine (it brings in users for other products), a retention anchor (it keeps users within a larger offering), a strategic bet (it exists to hold a position), or a utility (it must work and is rarely the reason someone chooses the company). The role changes what an improvement means. For a utility, a faster and more reliable core task is an improvement even if engagement falls. For a growth engine, a change that lifts sign-ups and lowers quality may still be wrong. State your assumption about the role in one sentence, and ask the interviewer to correct you.

Users. Name two or three [[user-segment]] groups by what they do, and which of them the product serves best today. Core users (most frequent, most dependent), casual users (occasional, lightly attached), and new users (still deciding) usually need different things, and an improvement for one often costs another.

Funnel. Walk the user's journey as a [[funnel]] (C05): how people arrive, start, complete the core task, return. Mark the steps where you would expect the largest loss and say why. A step with high traffic and low completion is a candidate; a step few people reach is not, however painful.

Metrics. Name the [[north-star-metric]] you believe the product follows, and one or two [[input-metric]] values. Say which look healthy and which do not, as hypotheses. If the interviewer gives numbers, use them and stop speculating.

Constraints. What cannot change easily: a large base of habitual users, integrations that others depend on, data stored in a certain format, a regulatory requirement, a platform rule, or hardware already in people's hands (C08.4).

Why it works. The five questions turn "improve X" into "improve X for this role, these users, this step," which is the form of a decision. The interviewer sees that you start from the product, not from your own idea.

When not to do it at length. If the interviewer gives the goal ("we want more people to finish setup"), take it, spend a minute on users and funnel, and move on. Do not rebuild a state the prompt already supplies.

C08.3 Choosing the improvement area: goal, segment, journey step

An open improvement prompt has three choices inside it: the goal, the segment and the step.

Goal. Choose one that fits the product's role: more of the right people complete the core task, return more often, pay, or recommend. State it as an outcome for users, then the business measure (C07's goal ladder, FW-C07-02). If the interviewer has already named a goal, use it.

Segment. Use C06 and C07: by need and behavior, with a criterion. In an existing product you can also segment by relationship to it: new, casual, core, lapsed.

Step. Pick the point in the journey where improvement is most worth having. Three tests, which FW-C08-03 turns into a table:

1. **Reach.** How many of the chosen segment arrive at this step?
2. **Pain or loss.** How badly does the step go for them: drop-off, repeated effort, workarounds, complaints?
3. **Headroom.** **Headroom** is the gap between how a segment performs at a journey step today and a realistic level it could reach, which bounds how much an improvement there can add. A step where 95% of people already succeed has little headroom, however annoying it is to the other 5%. A step where 40% succeed and similar products reach 70% has a lot.

A fourth test, fit with the goal, decides ties. The step with the most headroom is not always the one that moves the goal: improving a step that few people reach, or that does not lead to the outcome you chose, wastes the gain.

Hidden risk. Choosing by pain you can imagine, not pain you can size. A vivid frustration affecting 2% of users loses to a mild one affecting half of them, unless the 2% matter for a stated reason (a segment that pays, or a safety issue).

C08.4 Constraints of an existing product: legacy users, cannibalization, ecosystem effects

Three constraints deserve names, because naming them is half of handling them. FW-C08-04 is the scan.

Legacy users. A **legacy user** is a user (or customer) who depends on current behavior of the product (habits, integrations, stored data or contracts) so that a change creates cost for them even when it benefits others. Heavy users are often legacy users: they learned every shortcut. Enterprise customers are legacy users through their processes, training and integrations. The question to ask of any change is: who depends on how it works now, and what does the change cost them? Protect them with a path, not a promise: let them keep the old way for a period, migrate their data automatically, announce early, and measure their experience separately (FW-C08-07).

Cannibalization. **Cannibalization** is when a new feature or offer takes away use or revenue from an existing one of the same company, so that gross gains overstate net gains. A cheaper plan that mostly attracts existing full-price customers, or a new feed that mostly replaces time spent in the old one, adds little at the company level. Look for it by asking: where will this usage come from, new demand or an existing product? When the new thing is better for users, some cannibalization is the right outcome; the mistake is counting the moved use as growth.

Ecosystem effects. A change in one place has consequences elsewhere. *Across devices:* a feature on one device that others do not support creates inconsistent behavior, as when a note edited on one device cannot be opened on another. *With other systems:* integrations, partners and developers who built on current behavior (C19 treats platforms in full). *Between users:* in a product where people interact, a change for senders affects receivers, a [[network-effect]] working in reverse; a change that makes it easier to post can make the feed noisier for everyone who reads. *With rules and expectations:* platform policies, regulations and promises made to customers.

Regional Lens: India. In a mobile-first market, a large share of users are on mid-range or low-end Android phones, with limited storage and variable data connectivity. Inferred [unverified: author's synthesis; no source checked]. For an improvement, that turns into concrete constraints: app size, behavior on slow or intermittent connections, data use, and support for several languages. An improvement that adds weight to the app or requires a fast connection can be a regression for these users, so the legacy-user question has a regional form: who is on a device or connection that this change would break? Real-time account-to-account payments are widely used in India, notably through the Unified Payments Interface run by the National Payments Corporation of India Public [V: NPCI, UPI product overview], so a payment step designed around cards alone may miss how many users prefer to pay. This box adds a regional constraint; it does not replace the global method.

Why it works. The constraints are where an improvement answer shows maturity. Anyone can propose a feature; a candidate who says who would be hurt and how they would be protected has done the work of the job.

When not to dwell. Name the one or two constraints that matter for the chosen change; do not recite all three.

C08.5 Solution types: fix, add, remove, reposition

C07 listed kinds of solution (eliminate, automate, assist, reframe, connect). For an existing product, a second cut matters: what you do to the product. FW-C08-05 names four moves.

Move	What it is	Typical use	Main risk
Fix	Repair something that is broken or slow	A step with a high failure rate caused by a defect or poor wording	Treated as boring; the gain is real and often the largest per unit of effort

Move	What it is	Typical use	Main risk
Add	A new capability or option	A job users do elsewhere because the product cannot	Complexity; cannibalization; more to maintain
Remove	Delete or simplify	A feature that few use and many trip over; clutter	Someone depended on it (legacy users); you may not know why it was added
Reposition	Change what the product is for, who it is for, or how it is presented	The product is used differently from how it is described	Confuses current users; takes longer to show results

Most candidates reach for Add. Interviewers notice when you offer a Fix or a Remove, because those show you understand the cost of complexity. Before you recommend a Remove, check why the thing exists (the "fence" test in C08, mental models, section 9): the person who added it may have solved a problem you cannot see.

You can combine the two cuts. A "fix" can be an assist (better wording at a failing step) or an automation (filling in a field the user always enters). Use the move to describe what changes in the product and the kind to describe how it helps the user.

Why it works. The four moves widen the range beyond new features, and they show that you think about the whole product as something to be kept simple.

When not to use all four. If time is short, show two with a contrast (for example, a fix and an add) and say why you chose one.

C08.6 Reasoning about metric impact and sizing the opportunity

An interviewer often asks "how big could this be?" You do not need accuracy; you need a defensible chain and an honest range (C04). FW-C08-06 gives the ladder:

Opportunity = users who reach the step × baseline rate at the step × expected lift × value of one success, then haircuts.

Walk through an invented example. A recipe app has 2,000,000 weekly active users (invented). At a step called "save your first recipe," 30% of users arrive: 600,000. Baseline completion is 40%: 240,000 complete it. A fix is expected to raise completion to 46%: 276,000, which is 36,000 more completions per week. Haircut 30% for novelty wearing off and overlap with users who would have completed anyway: 25,200. Value of a completion: in the data, people who complete the step retain 20 percentage points better than those who do not (invented), but that is a correlation: people who save a recipe are probably more interested to begin with. Credit half of it as causal: $25,200 \times 0.20 \times 0.5 = 2,520$ additional retained weekly users, about 0.13% of the base (invented).

Three lessons. The number is small, and that is useful: it says whether this is worth a team's quarter. Each assumption is visible, so the interviewer can challenge one and you can change it. And the biggest haircut belongs to the step that is easiest to inflate: treating a correlation between completing a step and retention as the effect of the step is a common error ([proxy-metric], C05; and C14 on how to tell cause from correlation).

Check the result two ways. A second route: compare to what the metric did the last time something similar shipped, or to the gap between this step and a similar step in the product (the headroom). If the two disagree by more than a factor of two or three, one assumption is wrong ([sanity-check]], C04).

Say the range. "Between about 1,500 and 4,000 weekly retained users, which is under half a percent. The expected gain is small, so I'd want this to be cheap, and I'd look for a larger lever first."

Why it works. A sized answer shows that you can choose between ideas, not only generate them, and that you know that most improvements are modest.

When not to size at length. Give the chain in two or three sentences unless the interviewer asks for more; the structure matters more than the arithmetic.

C08.7 Answer architecture

Section 4 sets out the improvement spine with a time budget: clarify the product and goal, read the current state, choose the area, find the problem, generate moves, size and choose, measure, check risks. It reuses the pieces from C07 and adds the state read, the constraint scan and the sizing.

C08.8 Worked examples: consumer, enterprise, marketplace

Section 5 gives four worked improvements: a consumer messaging setting, an enterprise reporting tool, a marketplace, and a photo-library product. Section 11 adds seven questions with model answers, including two on devices and services made by Apple and one each set on a Google and a Meta product.

3. Frameworks

Seven frameworks, set out below. The goal ladder (FW-C07-02), the pain-point matrix (FW-C07-03), the option evaluation grid (FW-C07-05), the responsible-design check (FW-C07-06), the segment scorecard (FW-C06-02) and the success-metric template (FW-C05-06) are reused, not repeated.

FW-C08-01 · The improvement answer spine · Original

- **Inputs:** an improvement prompt about an existing product and 35 to 45 minutes.
- **Outputs:** an answer with eight visible steps: clarify, state, area, problem, moves, size, measure, risks.
- **Assumptions:** the interviewer will answer clarifying questions and may supply a goal or data.
- **Tradeoffs:** structure makes the reasoning scorable and can sound recited; the budget is a guide, not a quota.
- **Failure modes:** equal time on every step; no decision; treating it as a greenfield design; skipping the state read.

- **Worked example:** the messaging-settings improvement in Worked example 1, and the model answers in section 11.

Steps: **Clarify → State → Area → Problem → Moves → Size and choose → Measure → Risks**. The time budget is in section 4.

FW-C08-02 · The product state card · Original

- **Inputs:** a named product and what you know of it from use and public information.
- **Outputs:** a five-line card: role, users, funnel, metrics, constraints, each with the assumption you are making and what you would check.
- **Assumptions:** you can infer a product's role from how it is offered and used; the interviewer will correct you.
- **Tradeoffs:** a card fixes your assumptions early and can lock you into a wrong one; say so and invite correction.
- **Failure modes:** presenting guesses as facts; claiming internal knowledge; building a card so long it eats the time.
- **Worked example:** the state card for a photo-library product in Worked example 4.

Line	Question	Example answer (assumption)
Role	What is this product for, for the company and for users?	A utility that keeps users attached to the company's wider services
Users	Who uses it, and which group is served best today?	Heavy collectors served best; occasional users less so
Funnel	Where do people start, finish and return?	Strong capture, weak retrieval
Metrics	What does it follow, and what looks weak?	Retrieval success and return visits look weaker than storage
Constraints	What cannot change easily?	Years of stored data; a large base of habitual users

FW-C08-03 · The improvement-area selector · Original

- **Inputs:** a goal, two to four segments and the journey steps for each.
- **Outputs:** a table of candidate areas (segment and step) rated on reach, pain or loss, headroom and fit, with one or two chosen and a stated reason.
- **Assumptions:** you can estimate reach and headroom roughly from the funnel and comparisons.
- **Tradeoffs:** the table forces a choice and can imply false precision.
- **Failure modes:** choosing by the step you find annoying; ignoring headroom; rating everything high.
- **Worked example:** the selector for the marketplace in Worked example 3.

Segment and step	Reach	Pain or loss	Headroom	Fit with goal	Choice

Rule of thumb: choose the row where reach and headroom are both at least medium and fit is high. Name the row you rejected and why.

FW-C08-04 · The constraint scan · Original

- **Inputs:** a proposed change.
- **Outputs:** answers to four questions with a design response for each that applies: legacy users, cannibalization, ecosystem effects, and reversibility.
- **Assumptions:** you can reason about who depends on the current behavior and how it connects to other parts.
- **Tradeoffs:** thoroughness slows the answer; a short scan prevents a costly mistake.
- **Failure modes:** a list of constraints with no response; treating all constraints as equal; forgetting the quiet ones (integrations, other devices).
- **Worked example:** the scan for the enterprise reporting change in Worked example 2, and the sunset question in IQ-C08-06.

Check	Question	Typical response
Legacy users	Who depends on how this works today?	Keep the old way for a period; automatic migration; early notice; measure them separately
Cannibalization	Will the usage or revenue come from something we already have?	Count net, not gross; compare with a holdout
Ecosystem	What else does this touch: other devices, integrations, other users, rules?	Test across the set; version interfaces; staged rollout
Reversibility	Can we undo it, and how fast?	Release gradually; keep a rollback path (C17); treat irreversible steps with more evidence

FW-C08-05 · The four moves · Original

- **Inputs:** a chosen problem at a chosen step.
- **Outputs:** at least two moves of different types (fix, add, remove, reposition), each with how it works, why it helps, what it costs and what it risks, and one recommendation.
- **Assumptions:** the solution space includes changing or deleting what exists.
- **Tradeoffs:** range versus depth; a fix is cheap and modest, a reposition is large and slow.
- **Failure modes:** only adding; recommending a removal without checking why the thing exists; a reposition with no evidence that users see the product differently.
- **Worked example:** the fix versus add choice in Worked example 3, and the remove decision in IQ-C08-06.

Test before recommending a Remove: who uses it, why it was built, what happens to them, and how you would know if the removal was a mistake.

FW-C08-06 · The impact sizing ladder · Original

- **Inputs:** the chosen step, the product's scale and a proposed move.

- **Outputs:** a chain from users to value, with each assumption written, a haircut for novelty, overlap and correlation, a range, and a second-route check.
- **Assumptions:** order-of-magnitude values are enough to choose between ideas (C04).
- **Tradeoffs:** quick sizing guides priority and can be wrong by a wide margin; the value is in the visible assumptions.
- **Failure modes:** treating a correlation as the effect; no haircut; a single point estimate; sizing the step when the goal is elsewhere.
- **Worked example:** the recipe-app arithmetic in C08.6.

Chain: **reach × baseline × lift = extra successes; × (1 – haircut); × value per success (causal share only) = impact.** Then compare it to a second route and say the range.

FW-C08-07 · The winners-and-losers table · Original

- **Inputs:** a proposed change and the segments it touches.
- **Outputs:** a table of who gains, who loses, how each is likely to react, and a mitigation, plus the measure that will show the harm if it happens.
- **Assumptions:** you can name the segments and anticipate their reactions.
- **Tradeoffs:** attention to losers can slow or dilute a good change; ignoring them can damage trust.
- **Failure modes:** naming no losers; a mitigation that is only communication; averaging results across segments so the harm disappears.
- **Worked example:** the table for the messaging change in Worked example 1.

Segment	Gains or loses	Likely reaction	Mitigation	Measure

A change that wins on the average and hurts the most dependent segment is a decision to make knowingly, not by default.

4. Answer architecture

The improvement answer, for a 45-minute round

Step	Minutes	What to produce
Clarify the product and the goal	4	Two or three questions; the role and goal stated as assumptions
Read the current state	5	The five-line state card: role, users, funnel, metrics, constraints
Choose the area	7	Segments, journey steps, a table; one chosen with a criterion; what you give up
Find the problem	5	One or two specific pain points at the step, with evidence or how you would get it

Step	Minutes	What to produce
Generate moves	8	Two to four moves of different types (fix, add, remove, reposition)
Size and choose	6	A sizing chain with a range; the recommendation with its stated cost; the first version and what comes next
Metrics	5	Primary metric, an input, a guardrail, a counter metric
Risks	3	Winners and losers; the constraint that matters; what would make you reverse
Slack for interviewer questions	2	

Pacing rules. If the interviewer gives you the goal or the segment, move the saved time to the problem and the moves. If you run behind, drop to two moves, a one-sentence size and one risk, but keep the recommendation and the losers. Treat questions as steering and rejoin the spine.

What to clarify first. The product and the platform (the version the interviewer has in mind), any goal they want, and any constraint. Ask two or three and state assumptions for the rest. A useful opener: "I'll assume we are improving the existing mobile experience for current users, and the goal is more people completing the core task. Tell me if you want something else."

Where to signpost. After clarifying: "I'll cover what the product is for, where to focus, the problem, a few options, a size and a recommendation." Before choosing: "I'll pick by reach and headroom." Before recommending: "My deciding criterion is X, and I'll say who loses."

How to close. Two sentences: "I recommend [move] for [segment] at [step], to fix [problem], and I'd measure success by [metric]. It could hurt [segment], which I'd protect with [response], and I'd reverse if [condition]."

Type: the goal is given

"Improve retention for X." Skip the goal choice, use the saved minutes on the segment and the step: where do the people who leave leave from? Use cohort thinking (C05) to say what you would look at.

Type: which part of the product to improve

"What would you improve about X?" Spend the first half on the selector (FW-C08-03), because the choice is the answer. Name two areas you considered and why you rejected the one you did.

Type: a competitor's or an unfamiliar product

Say that you know the product only from public information and use. State assumptions as such, label them, and build the answer on user needs and the funnel logic, not on claimed insider facts. Invite correction.

Type: the short version (20 to 25 minutes)

Keep every step and halve the time: one segment, one step, two moves, a one-line size, one metric, one loser. Protect the recommendation and the losers.

Common interviewer interventions, and what to do

Interviewer says	What it means	What to do
"What is this product for?"	You skipped the role	State the role for company and users in a sentence
"Why that part of the product?"	Your choice is unclear	Give reach, pain and headroom, and what you rejected
"How big could this be?"	They want sizing	Give the chain with a range and the biggest haircut
"What about the people who like it as it is?"	You skipped the legacy users	Name them, the cost to them and a protection
"Wouldn't this just move usage from somewhere else?"	Cannibalization	Say where the usage comes from and count net
"Do you need to build anything?"	They want a fix or a removal	Offer the cheaper move and compare
"How would you know it worked?"	They want a metric	Use the C05 template with a guardrail and counter metric

5. Worked examples

All four examples are **Illustrative**. Where a real company's product type is the setting (examples 1 and 4), nothing here is a claim about that company's actual numbers, roadmap or decisions; every figure is an invented assumption. Each follows the spine in FW-C08-01 and is written as notes you could say aloud. A spoken answer is shorter.

Worked example 1 — Consumer: improving group chats in a large messaging app (Illustrative)

Setting: a messaging product of the kind Meta operates. No fact about any real product is claimed.

The prompt. "How would you improve group chats in a large messaging app?"

Clarify and goal. I'd ask whether we are improving the existing mobile app for current users and what the company wants. Assume yes, and the goal is that group chats stay valuable to the people in them, so that people keep taking part. Goal ladder (FW-C07-02): the company wants groups to keep people on the product; the product goal is steady participation in groups; the user's goal is to stay in touch with the people in the group without drowning.

State card (FW-C08-02).

Line	Assumption
Role	A core product and a retention anchor: people stay because their groups are there, which is a [[network-effect]]
Users	Heavy group users (family, friends, neighborhood, workplaces); casual members of busy groups; group creators and admins
Funnel	Create or join, first messages, steady participation, then mute or leave
Metrics	Weekly active groups; messages per active member; mute and leave rates, which I'd treat as early warnings of decay
Constraints	A change for senders affects receivers; members in groups of 200 or more rely on current behavior; every device must behave the same

Choosing the area (FW-C08-03).

Segment and step	Reach	Pain or loss	Headroom	Fit with goal	Choice
Members of large groups (50 or more), keeping up with a busy chat	Medium: 20% of users are in one (invented)	High: 45% mute within 30 days of joining (invented)	High	High	Chosen
Group creators, inviting people	Low	Medium	Medium	Medium	Rejected: fewer people affected
Members of small groups, reading and replying	High	Low: 5% mute (invented)	Low	Medium	Rejected: little headroom

Problem. After a burst of messages, a member of a large group cannot find what matters to them (a mention, a reply to them, a plan with a time). They mute the group, which removes them from the conversation while they remain in it. The workarounds are evidence: leaving the group, or messaging a friend "what did I miss?" I'd confirm by interviewing members who muted, and by checking what share of mutes follow a day with more than 100 messages.

The four moves (FW-C08-05).

Move	Idea
Fix	Make mentions and replies to me always notify and be easy to see, since many members do not realize a reply was to them
Add	A catch-up view at the top of a chat: mentions of me, replies to me and plans with times, in order (rule-based, not a summary written by a model)
Remove	Limit the "notify everyone" action in groups over 100 to admins
Reposition	Treat large groups as communities with sub-topics

Sizing (FW-C08-06). Assume 4,000,000 people a month join a large group (invented). At a 45% mute rate within 30 days, 1,800,000 mute. If the catch-up view cuts muting by 10% in relative terms, the mute rate becomes 40.5%: 1,620,000 mute, which is 180,000 fewer.

Haircut 40% for people who never see the view and novelty: 108,000. Members who do not mute are still active after eight weeks at 70%, against 25% of those who mute (invented), but that is a correlation, so credit 40% as causal: $108,000 \times (0.70 - 0.25) \times 0.4 = 19,440$ more active members from each month's joiners, about 0.5% of 4,000,000. Second route: the mute rate falls 4.5 points, and an active-share difference of 45 points gives $4.5 \times 45\% \approx 2$ points of the joiners before haircuts; after haircuts that is about 0.5 points, which agrees. The gain is modest, so the first version should be cheap.

Recommendation. "I recommend the Fix and the Add together for the first version: reliable mentions and replies, and a rule-based catch-up view for large groups. It addresses the cause of muting by giving up the more ambitious reposition into communities. Next I'd add plan reminders, and I'd revisit the community idea if muting stays high among very large groups." The remove is held back because admins of big groups use the notify-everyone action, and I don't know why each does.

Winners and losers (FW-C08-07).

Segment	Gains or loses	Likely reaction	Mitigation	Measure
Members of large groups	Gain	Use the view; mute less	Show it only after a gap, not always	Mute rate; eight-week active share
Senders of general messages (not a mention or reply)	May lose attention: their messages are not in the "for me" view	Post less, or tag more	Keep the full chronological view as the default and the catch-up view as an addition	Messages per sender; replies per message
Admins who use notify-everyone	Unaffected in the recommendation	None	None	None

Metrics. Primary: the share of new large-group members still active in the group after eight weeks. Input: the share who open the catch-up view at least once in two weeks. Guardrail: messages sent per active member in large groups, which must not fall. Counter metric: replies received per message for senders who did not mention anyone. Reverse if replies per message fall by more than 5% (invented).

Worked example 2 — Enterprise: improving reports in a project-management tool (Illustrative)

The prompt. "Customers say the reports in our project-management tool are poor. How would you improve them?"

Clarify and goal. Assume a tool sold per seat to companies, and the goal is renewals and expansion. The buyers are department heads, the daily users are team members, and the report readers are team leads and managers. I'd say that the buyer, the administrator and the daily user differ, and that reports mostly serve the first and the readers above them.

State card. *Role:* revenue engine, sold by seat. *Users:* team leads (build reports), executives (read them), team members (feed the data). *Funnel:* tasks are entered, then reviewed, then summarized into a report for a manager each week. *Metrics:* weekly report runs, share of accounts that use reports weekly, renewal rate. *Constraints:* customers have built weekly routines, scripts and dashboards on the current export and the programming interface.

Choosing the area.

Segment and step	Reach	Pain or loss	Headroom	Fit	Choice
Team leads assembling the weekly status report	High: about half of accounts (invented)	High: about two hours a week by hand (invented)	High	High: reports drive renewals	Chosen
Executives reading dashboards	Medium	Medium	Medium	Medium	Rejected for now
Admins configuring permissions	Low	Medium	Medium	Low	Rejected

Problem. Leads export tasks to a spreadsheet and build the status report by hand each week, because the built-in reports are slow to load and show wrong totals when tasks are reassigned. I'd check the complaint categories in support tickets and watch a lead build one.

The four moves. *Fix:* correct the totals and the load time, which is cheap and earns trust. *Add:* a scheduled status summary that builds itself and arrives by email on a chosen day. *Remove:* retire the old custom report builder, used by 3% of accounts (invented), because it is hard to maintain; I would not do this first, since those accounts are likely the most dependent. *Reposition:* from a task tracker to a status-communication tool.

Constraint scan (FW-C08-04).

Check	Question	Response
Legacy users	Who depends on today's export and report behavior?	Accounts with scripts that read the export columns. Keep every existing column and name; add new fields instead of changing old ones
Cannibalization	Does it take usage from something else?	It replaces manual spreadsheet work, not product usage, so it adds
Ecosystem	What else touches this?	Integrations and the programming interface. Version the export, announce six months ahead, and track calls to the old format
Reversibility	Can we undo it?	The totals fix and the new summary are reversible behind a flag; a changed data format is not, so the format stays

Sizing. Assume 8,000 accounts (invented), 4,000 using reports weekly and 4,000 not. The scheduled summary moves 20% of non-users into weekly use: 800 accounts. Haircut 30% for accounts that would have started anyway: 560. Time saved: 560 accounts × 6 leads (invented) × 1 hour a week × 48 weeks ≈ 161,000 hours a year. Renewal: weekly users renew at 94% against 86% for others (invented), a correlation, so credit 30%: 560 × 0.08 ×

0.3 $\approx$ 13 more renewed accounts, and at \$20,000 a year each (invented), about \$270,000. The time saved is large for customers and the renewal effect is modest, so I'd present the value as customer time and risk reduction, and not promise a revenue number.

Recommendation. "I recommend the Fix and the scheduled summary, keeping every existing export column unchanged, which gives leads their two hours back and removes the most common complaint, by giving up a bigger redesign of reporting. Next I'd add sharing to executives, and I'd retire the old builder only after finding out why those accounts use it." Metrics: primary, the share of accounts using a scheduled report weekly after 90 days; input, time from opening reports to sending the summary; guardrail, calls to the old export format (a rise would mean broken scripts), and support tickets about reports; counter metric, the share of scheduled summaries sent but not opened, which would show an automated output nobody reads. Reverse the summary if open rates fall below a threshold set in advance.

Worked example 3 — Marketplace: increasing repeat bookings in a home-services marketplace (Illustrative)

The prompt. "Improve a marketplace where customers book home services such as cleaning and repairs."

Clarify and goal. The goal given by the interviewer: more repeat bookings. I'd note it is a two-sided product, so a change for customers affects providers. State card: *Role*: a revenue engine that takes a commission. *Users*: first-time customers, returning customers, and providers of varying experience. *Funnel*: search, request, provider confirms, service, review, rebook. *Metrics*: the share of first-time customers who book again within 60 days (invented baseline 30%), commission, provider cancellation rate. *Constraints*: providers depend on the booking flow for income; there is a risk of customers and providers moving off the platform to rebook directly.

Choosing the area.

Segment and step	Reach	Pain or loss	Headroom	Fit with repeat goal	Choice
First-time customers, request to confirmed booking	High	High: 35% of requests unconfirmed after 24 hours (invented)	High	Medium: it serves the first booking, not the second	Rejected for this goal; a separate effort
Returning customers, rebooking after the first service	Medium: about a third reach this stage (invented)	Medium: they must search again and cannot see the same provider's availability	Medium	High	Chosen
Providers, setting quotes	Low	Medium	Medium	Low	Rejected

I name the rejection: the confirmation step has a larger loss, but it is not the step that decides a repeat booking, and a customer whose first request fails never gets to a second. If the goal had been total bookings, I would have chosen it.

Problem. After a good first service, a customer who wants the same provider again has to search from the start, and cannot see when that person is free. Many will message the provider directly, which takes the booking off the platform. The workaround is the evidence.

The four moves.

Move	Idea	Note
Fix	Put "Book again" on the completed booking, with the provider's next available times	Cheap; touches one screen
Add	Recurring bookings (every two weeks, same provider)	Larger; needs provider calendars
Remove	Remove the post-service rating prompt that currently blocks the screen the customer sees right after the service	Easy; ratings are valuable, so move it, don't delete it
Reposition	From "find a pro" to "your home's regular helpers"	Large and slow

Cannibalization and second-order effect. Easier rebooking reduces new-provider discovery: customers book their favorite and new providers get fewer first jobs. That is a [[second-order-effect]] on the supply side, so a counter metric should track it. It also reduces the incentive to leave the platform, which is a benefit that the headline metric will not show.

Sizing. Assume 200,000 first-time customers a month (invented). At 30% rebooking within 60 days, 60,000 rebook. The fix lifts it to 33%: 66,000, which is 6,000 more. Haircut 30% for customers who would have rebooked anyway: 4,200. At an average booking of \$80 and a 20% commission (invented), $4,200 \times \$80 \times 0.20 = \$67,200$ a month. Second route: the one-screen fix is cheap, and a three-point lift on a 30% base is plausible for a button moved from hidden to prominent; I'd accept the number as an order of magnitude and look at the cost.

Recommendation. "I recommend the Fix and the moved rating prompt in the first version, which gives returning customers a one-tap path to the same provider by giving up recurring bookings for now. If rebooking rises and customers ask for recurring service, I'd build the Add next. I'd switch to the larger move sooner if interviews show that most repeat customers want the same time every week." Metrics: primary, the share of first-time customers booking again within 60 days; input, taps on "Book again"; guardrails, provider cancellation rate and the average rating, which must not fall because the prompt moved; counter metrics, the share of first-time jobs going to providers with fewer than five reviews (new-provider starvation) and off-platform contact attempts.

Worked example 4 — Photo library: improving how people find their photos (Illustrative)

Setting: a photo-library product of the kind Google operates. No fact about any real product is claimed.

The prompt. "How would you improve a photo-storage and sharing product?"

Clarify and goal. I'd assume the existing mobile and web product, and a goal of making the product the place people go when they want a photo. State card (FW-C08-02): *Role:* a utility and retention anchor for a wider set of services, with storage plans as a possible revenue

link. *Users*: heavy collectors with years of photos and tens of thousands of items, casual users with a few hundred, and families sharing albums. *Funnel*: capture and back up, organize, find, share, revisit. The product is likely strong at the first step because it is automatic, and weakest at finding a specific photo years later. *Metrics*: backup volume (healthy), search use and search success (my hypothesis: weaker), shares. *Constraints*: years of stored data, a large base of habitual users, and privacy sensitivity of the content.

Choosing the area.

Segment and step	Reach	Pain or loss	Headroom	Fit	Choice
Heavy collectors, finding a specific photo	Medium: 40% of monthly users search (invented)	High: 45% of searches end without opening a result (invented)	High	High	Chosen
Casual users, first backup	High	Low	Low	Medium	Rejected
Families, sharing albums	Medium	Medium	Medium	Medium	Rejected for now

Problem. A person looking for "that dinner in the spring" scrolls by date, tries a few words, and gives up or asks a relative. Search on the content of photos and on everyday phrases is the weak point. Evidence I'd look for: searches with no opened result, repeated reformulations within a minute, and what people do next (scrolling, asking others).

The four moves. *Fix*: make search understand everyday words and dates, and show why a result matched. *Add*: automatic albums for events. *Remove*: remove the manual album clutter for casual users, which power users rely on, so I'd not do it without a check. *Reposition*: from storage to a "memories" product.

Constraint scan. Legacy users: people with carefully built manual albums, who must keep them. Ecosystem: the same library appears on phones, tablets and the web, so search must behave alike. Responsible design (FW-C07-06): automatic labeling can be wrong, and some people and moments should not resurface. A "memories" or "on this day" style feature can show a painful moment (a death, a breakup), and the design response is to let users hide specific people and dates, and to make unwanted resurfacing easy to turn off. Privacy: grouping photos by faces is sensitive, so it is opt-in and the user can see and delete the groupings.

Sizing. Assume 50,000,000 monthly users (invented). 40% search: 20,000,000. Search success rises from 55% to 60%: 1,000,000 more successful searchers a month. Haircut 30%: 700,000, which is 3.5% of searching users. I would stop the chain there, because the link from a successful search to a company outcome (storage plan upgrades, return visits) is weak and mostly speculation, and I'd say so. Present it as an experience gain and measure return visits afterward, and not promise revenue.

Recommendation. "I recommend the Fix for the first version: better search on everyday words and dates, with a short reason for each match. It addresses the largest observed loss by giving up the automatic albums and the repositioning, both larger. Next I'd add events,

with the controls above. I'd switch to the Add if interviews show that people don't search but prefer to browse collections." Metrics: primary, search success (a result opened or shared within two minutes); input, the share of queries that return at least one result; guardrails, the rate of wrong-label complaints and the privacy-control usage; counter metric, the share of users who turn off suggestions, which would show that a feature meant to help is unwelcome.

6. Baseline, Strong, Standout

Prompt: "How would you improve an online grocery-delivery app?"

Baseline answer

"I'd add more features: a recipe section, voice ordering, AR to see how the food looks, and loyalty rewards. I'd also make the app faster and prettier."

What is wrong: a list of additions with no statement of what the product is for, who uses it, or where it falls short. Nothing is chosen, sized or measured, and no one is protected.

Strong answer

"I'll assume the goal is more weekly orders from people who already use the app. The app is a revenue engine and its core customers are households that order every week, so I'd start there. The funnel is browse, build a basket, check out, receive and review, then reorder. I'd expect the largest loss and effort at building the basket, because a weekly shopper buys many of the same items.

"Candidate areas: new shoppers at their first order, regular shoppers at rebuilding the basket, and everyone at delivery slots. I'd choose regular shoppers rebuilding the basket: they are the most frequent, the pain repeats every week, and there is headroom because they rebuild 20 items by hand. The problem is that it takes them about 12 minutes to rebuild the same basket (an assumption to verify).

"Moves: a fix to make 'order again' easier to find; an add, a prefilled basket of the usual items; a removal of promotional carousels that sit between the user and the basket; a reposition into a 'weekly staples' service. I'd recommend the prefilled basket with the usual items, starting from the last order. Metrics: orders per active household per month, with the time to build a basket as an input."

Standout answer

Everything in the Strong answer, plus:

"Prefilling the basket has a cost that I'd name: browsing is where some of the unplanned items come from, so faster reordering could reduce basket size. That is cannibalization of discovery, so I'd put basket value per order as a counter metric and look at net order value, not only order frequency.

"Substitutions are a legacy-user issue. A prefilled basket assumes the items are available, and when something is out of stock, regular shoppers have strong preferences about what replaces it. A prefilled basket that surprises them damages trust, so the design needs to show availability before checkout and let them set substitution rules once.

"On sizing: if 60% of 1,000,000 weekly households (invented) rebuild a basket by hand and the prefill saves 8 of 12 minutes for half of them, that is real time saved, but the revenue effect depends on whether it raises orders per month, which I would test, not assume. Households that already order weekly have little room to order more, so the gain is mainly among those who order every few weeks and find the effort a barrier; I'd target that group in the test.

"The risk to others is the picking and delivery side: a prefilled basket concentrates demand on staple items and delivery slots at peak times, so I'd watch slot availability. I'd change my mind if interviews showed that the effort in the basket is deciding what to buy, not typing it, in which case the prefill would not help and the real lever is meal planning."

What moves an answer up a band

From Baseline to Strong	From Strong to Standout
States the product's role, users and funnel	Names cannibalization and the legacy-user issue for the change
Chooses a segment and a step by a criterion	Sizes the gain, and says where the chain is weak
Offers more than one type of move and recommends one	Names the second-order effect on other sides of the product
Gives a metric with an input	Adds a counter metric and the condition for reversing

7. Company lens

All claims describe what companies say publicly or what candidates commonly report, not insider knowledge. Interview formats change [as of 2026-10] .

Company lens: Google [as of 2026-10]. Candidates commonly report improvement prompts about Google's own products, often at very large scale and across many countries Inferred [unverified: candidate-reported] . Google has long published a stated philosophy that starts with focusing on the user Public [V: Google, Ten things we know to be true] . Practical consequence: say who the product serves at scale, include users on slower devices and connections, and note that a small percentage change affects a very large number of people. Appendix A has briefs on eight Google products to practice against.

Company lens: Amazon [as of 2026-10]. Amazon publishes Customer Obsession and Invent and Simplify as leadership principles Public [V: Amazon's Leadership Principles, as cited by several summaries]. Candidates commonly report that improvement answers are expected to begin with the customer's problem and to show a bias for simplicity Inferred. Practical consequence: offer a removal or a simplification as one of your moves, and say what the customer would notice.

Company lens: Meta [as of 2026-10]. Candidates commonly report that improvement prompts at Meta are paired with questions about tradeoffs between engagement and wellbeing or integrity, and with questions on how you would measure a change Inferred [unverified: candidate-reported]. Practical consequence: pair each change with a counter metric, and consider what a change does to people on the other end of an interaction.

Company lens: Microsoft [as of 2026-10]. Microsoft's products often serve large enterprise customers with long-lived processes and integrations, and the company is widely described as valuing backward compatibility Inferred [unverified: candidate-reported]. Practical consequence: separate the buyer, the administrator and the daily user, and show that you protect existing customers when you change behavior.

Company lens: Apple [as of 2026-10]. Apple designs hardware, software and services together and publishes platform guidelines and a privacy position Public [V: Apple Human Interface Guidelines (developer.apple.com) and privacy pages]. Third-party guides report that improvement prompts at Apple concern its own devices and weigh consistency across them and attention to detail Inferred. Practical consequence: check each change across the user's other devices, say what stays on the device, and keep a change small if a larger one would make the experience inconsistent.

Company lens: Netflix [as of 2026-10]. Netflix publishes material on its culture and on its approach to experimentation Public [V: Netflix culture memo, June 2024; Netflix Tech Blog on experimentation]. Candidates commonly report data-informed improvement questions Inferred. Practical consequence: say what you would test, with which metric and how long, and be explicit about novelty effects that fade.

Company lens: Stripe [as of 2026-10]. Stripe documents a dated approach to versioning its programming interface so that existing integrations keep working when behavior changes `Public [V: Stripe API versioning documentation and engineering blog]`. Candidates report developer-facing improvement prompts `Inferred`. Practical consequence: for products that other people build on, treat compatibility as a feature: version the change and give customers time.

8. Common mistakes

#	Mistake	How to detect it	Fix
1	Treating the product as new	Your first sentence is a feature	State the role, users and funnel first
2	Not knowing what the product is for	You cannot say how it matters to the company	Name the role and ask to be corrected
3	Improving what you personally dislike	Your evidence is your own annoyance	Segment, reach and headroom
4	Choosing a step with no headroom	Most people already succeed there	Compare loss and what a good level would be
5	Only adding features	All your moves are new things	Offer a fix and a removal beside the add
6	Removing without asking why it exists	You call something "clutter"	Apply the fence test: who uses it and why was it built
7	No sizing	You cannot say whether it is worth a quarter	Give the chain, haircut it and say the range
8	Treating correlation as the effect	You credit all of the retention gap to the step	Credit part of it, name the selection effect
9	No losers	Everyone gains in your answer	Fill in the winners-and-losers table
10	Averaging over segments	A good average hides harm to the core group	Measure the most dependent segment separately

9. Mental models applied

Four models, each tied to a decision in this chapter. `[[local-optimum]]` comes from C02 and `[[goodharts-law]]` from C05.

Chesterton's fence (`MM-chestertons-fence`). Before removing something, learn why it was put there. A feature or rule that looks pointless may be protecting someone you cannot see: an enterprise account's script, a safety check, a legal requirement. In an interview, the fence test means asking "who uses it and why was it built?" before a removal, and it signals judgment that the cost of complexity is balanced against the cost of breaking something.

The local maximum (MM-local-optimum, see [[local-optimum]], C02). Repeated small improvements to a step can raise a metric to the best a design can reach without making the product better for the person. A checkout flow can be tuned until little more can be gained while the real problem is that people do not want what they are checking out. When the improvements you can think of have small sizes, say so, and consider a larger move such as a reposition.

Loss aversion (MM-loss-aversion). People feel losing something familiar more strongly than gaining something of equal value, which is why a change that is better on average can feel worse to the people who lose a habit. It is the reason legacy users react strongly and the reason for a gradual path: let them keep the old way for a time, and show what they gain before you take away what they have.

Goodhart's law applied (MM-goodhart, see [[goodharts-law]], C05). When a measure becomes a target, it stops being a good measure. An improvement that raises a step's completion by making it easier to tap through, or that pushes notifications until sessions rise, can raise the number and lower the value to the user. Name the counter metric whenever you name the target.

Engineering, design and data-science perspectives

- **Engineering.** An existing product carries history: older code, data formats and integrations that make a change cheaper or costlier than it looks. Ask early what a change touches, what cannot be changed without a migration, and how the change can be released gradually (C17). Listen when an engineer says "that field is used by something else."
- **Design and research.** A designer knows the existing patterns that users have learned and the consistency the product depends on. Ask how much of the change users must relearn, and watch current users attempt the old and the new flow; a change that tests well with new users can still break a habit in core users.
- **Data science.** Existing products have data, and the question is what it can and cannot show. Ask for the funnel and the segment breakdown before choosing, agree what counts as success and the counter metric, and check whether a difference between users of a feature and non-users comes from the feature or from who chose it (C14 on experiments).

Checklist: before you recommend a change to an existing product

1. Did I say what the product is for, for the company and for users?
2. Did I choose a segment and a journey step by reach, pain and headroom, and say what I gave up?
3. Is the problem specific, with the evidence I have or would collect?
4. Did I offer more than one type of move (fix, add, remove, reposition)?

5. Did I size the gain with a visible chain, a haircut and a range?
6. Did I say who loses, what protects them and how I would see harm?
7. Did I check cannibalization, other devices and anything that depends on current behavior?
8. Does the metric have a guardrail and a counter metric, and did I say what would make me reverse?

10. Case studies

CS-C08-01 · The tool-lending marketplace and the unanswered request · Illustrative · Beginner

Fictional composite. Every number is invented (invented).

Situation. A neighborhood tool-lending marketplace lets people list tools such as drills and ladders and lets neighbors borrow them. In a typical month, borrowers send 40,000 requests (invented), and the team notices that 11,000 loans are completed (invented), which is 27.5 per 100 requests. Management wants more completed loans and proposes to offer damage insurance on every loan.

Constraints. A product team of three, a fixed quarter, owners who are unpaid volunteers whose goodwill is the scarce resource, and no budget for incentives.

Data. The funnel for the month (invented): 40,000 requests sent, 34,000 viewed by an owner, 18,000 with a reply, 12,000 accepted, 11,000 completed. In 10 interviews with owners, six say they saw a request and meant to reply later, then forgot; four say they were unsure about the borrower and the return date. Reply timing shows that 62% of replies come within two hours of a push notification, but only 30% of owners have push notifications enabled (invented).

Problem definition. The loss at each step: 6,000 requests are never viewed (15%), 16,000 viewed requests get no reply (47%), 6,000 replied requests are not accepted, and 1,000 accepted loans are not completed (8%). The largest loss and the most headroom are between viewing and replying. As an observable pain point: "An owner sees a request, cannot tell quickly whether to trust the borrower or when the tool would come back, and postpones replying until the request goes stale."

Analysis. Using the selector (FW-C08-03), request-to-view has medium headroom and a cheap fix (push notifications), view-to-reply has the largest loss, and accepted-to-completed has little headroom. The interviews explain the loss: effort (writing a reply) and doubt (who is the borrower and when is the return). Insurance would address damage anxiety, which only some owners raised, and would cost money the team does not have, so it fits the problem poorly. Sizing the chosen step: 34,000 viewed requests, 18,000 replies (52.9%). A fix that lifts replies to 60% gives 20,400, which is 2,400 more.

Haircut 30%: 1,680. Two thirds of replies become accepted loans (12,000 of 18,000): 1,120 more accepted, and 11 of 12 complete: about 1,030 more completed loans, a rise of 9.3% on 11,000. The alternative, a notification fix, reaches the 6,000 unviewed requests. If it gets a quarter of them viewed (1,500), and those owners reply at a lower rate, say 40%, the result is 600 replies, about 400 accepted loans and 370 completed, which is smaller.

Options.

Option	Optimizes	Costs
A. Fix notifications: ask owners to enable push when they list a tool	Requests viewed	Cheap; smaller gain
B. Add one-tap replies ("Yes, pickup after 6") and show the borrower's history and the return date on the request	Replies	About three weeks of work; a risk of low-effort replies
C. Add damage insurance on every loan	Trust at handover	Real money; solves a problem only some owners named
D. Do nothing until more data	Certainty	A quarter of delay

Tradeoffs. B has the largest sized gain and aims at the cause owners described, and it gives up the more thoughtful tone of written replies; owners can still write one. A is cheap and can be done in the same sprint, though its gain is smaller. C may matter later, once replies are no longer the bottleneck.

Decision. Build B first, and ship A in the same quarter as a small change. The primary metric is completed loans per 100 requests (baseline 27.5). Guardrails: the share of loans completed (the 91.7% accepted-to-completed rate must not fall) and damage reports per 1,000 loans. Counter metric: the share of one-tap replies that are refusals, which shows whether faster replies are simply faster no's.

Outcome. In this fictional scenario the reply rate rose from 52.9% to 58%, which at the same acceptance and completion rates gives about 12,050 completed loans, and completed loans per 100 requests rose from 27.5 to 30.1 (invented). One-tap refusals made up 22% of replies (invented), which the team treated as healthy because borrowers learned sooner and asked someone else. Damage reports did not change (invented). The team deferred insurance and asked a sample of owners whether it would change their willingness to lend.

Alternative outcomes. If replies had risen but completions had not, the doubt at handover would have been the real barrier, and insurance would have moved up. If one-tap replies had reduced the information exchanged and damage reports had risen, the borrower history would have been shown more prominently. If owners had disliked the change, the legacy-user check (owners who liked writing replies) would have called for keeping a prominent text option. The lesson: find the step with the most loss and headroom, size the options against each other, and take the cheap fix along with the larger move.

CS-C08-02 · Twitter, 2017: doubling the character limit · Verified · Advanced

Sources (types): Twitter's own product blog posts announcing the test ("Giving you more characters to express yourself," 26 September 2017, read through excerpts) and the rollout ("Tweeting Made Easier," 7 November 2017, read in full), both primary, the company's own

account; contemporaneous news coverage (Fortune, NPR, GeekWire, The Register); a TechCrunch report of company data (30 October 2018); and NPR's 2011 interview with co-founder Biz Stone for the origin of the 140-character limit. Twitter was later renamed X (the company's blog pages now carry the X Corp. name) [as of 2026-10]; this case uses the name in use at the time. The analysis is the author's. Re-read in October 2026.

Situation. Twitter had limited posts to 140 characters since its launch in 2006, a limit that co-founder Biz Stone said came from the 160-character length of a text message, with 20 characters left for the username [V: NPR, 2011]. In September 2017 it began testing a 280-character limit with a group of users, for all languages except Chinese, Japanese and Korean. On 7 November 2017 it rolled the longer limit out to nearly all users, with those three languages excluded.

Constraints. The brevity of posts was a defining trait of the product, and many users reacted to the test with concern that timelines would fill with long posts, as reported at the time. The company said it wanted to ease the frustration of editing posts to fit while keeping the speed and brevity that defined the product. Some people in the test were on older versions of the apps that could not send longer posts, which the company noted beside its English-only chart of the test [V: Twitter blog, 7 November 2017].

Data. As reported by the company: in English, about 9% of posts hit the 140-character limit, compared with about 0.4% of posts in Japanese; the same post said that the most common post length was 34 characters in English and 15 in Japanese [V: Twitter blog, 26 September 2017]. In the test group, with the longer limit, only about 1% of English posts hit the cap, only 5% of all posts exceeded 140 characters and only 2% exceeded 190. The company said that people with more room posted more, received more engagement (likes, reposts, mentions), gained more followers and spent more time on the product, and that participants told the company a higher limit made them feel more satisfied with how they expressed themselves, with their ability to find good content and with Twitter overall [V: Twitter blog, 7 November 2017; company-reported, figures not published]. It also said early use of the full limit was high because the feature was new and then settled. A report in October 2018, based on company data, said that about 1% of posts reached the 280-character limit, about 12% were longer than 140 characters, and about 5% exceeded 190 (for English posts; 6% and 3% globally) [V: TechCrunch, 30 October 2018; company data].

Problem definition. *The author's reading.* An observable pain point: in English, a person writes a post, finds it over the limit, and spends time cutting it or gives up. The company's 9% figure sizes it: nearly one post in ten hit the cap.

Analysis. *The author's analysis.* Read through this chapter's frameworks, the decision is a clean use of headroom. The area was the compose step. For English, 9% of posts hitting the cap means a defined group of people losing effort at one step. For Japanese, 0.4% hit the cap, because the language carries more per character, so there was little headroom and the company left it alone. This is a segment choice made by headroom, and the exclusion followed from the data.

The legacy-user constraint was the brevity that long-time users valued, and the test answered it with a guardrail: the share of posts above 140 characters, which was small. The ecosystem constraint was older apps that could not send longer posts, which made the test

group's behavior look more conservative than it would be once all apps supported it. The test also shows a measurement lesson. The 5% figure for the test period was lower than the 12% reported a year after rollout. Possible reasons, which are inference, include users changing their habits over time, a test group that included users on older apps, and a sample that differed from the full user base. In any case a test result is a forecast and not the final outcome, and a guardrail such as "few posts above 140" should be rechecked after launch.

The company-reported gains in engagement, followers and time spent come from the company's own test and cannot be checked independently from public sources. They were reported as the result of the test, so they say little about long-run growth. A reader should treat them as the company's justification, not as proof of cause.

Options. *The author's reconstruction of choices a team could have faced; not a record of the company's deliberations.*

Option	Optimizes	Costs
A. Keep 140 and change nothing	The product's identity and consistency	The friction for the roughly one in ten English posts that hit the cap
B. Raise to 280 for languages where the cap binds	Compose friction, with limited change to timelines	Risk to brevity, which the test could check
C. Allow very long posts	Expression	A larger change to what the product is, and to timelines
D. Keep 140 and offer a long-form or thread option	Identity preserved for the short post	Compose friction stays; a second place to write

Tradeoffs. B gives up strict brevity in exchange for removing a measured source of friction, and it is reversible in the sense that the limit can be changed again. C changes the product's character, and A leaves the measured pain. D adds a feature, and adding complexity is the classic reflex that the four-moves framework warns about.

Decision. As documented, the company tested 280 for about six weeks and then released it to nearly all languages, leaving Chinese, Japanese and Korean at 140.

Outcome. Reported: the share of English posts hitting the cap fell from 9% to 1%, and the test showed that most posts stayed short. A follow-up in October 2018, reported by TechCrunch from company data, said that the most common English post length was 33 characters against 34 under the old limit, that about 1% of English posts hit the 280-character limit and that 12% of English posts were longer than 140 characters (6% of posts globally), with findings consistent across seven languages [V: TechCrunch, 30 October 2018]. The company's data did not settle whether the change improved long-run growth; the sources used here do not say.

Alternative outcomes. *Inference.* Had the share of long posts been high in the test, timelines would have been denser and the guardrail would have argued against the change. Had long-time users left in large numbers, the change would have been costly in a way the average hides, and a separate measure of the heaviest users would have shown it. A candidate analyzing this case should also note the survivorship problem: a change that

worked is easy to explain afterward, and the same facts would have been read differently if it had failed. The method to carry away: size the pain at the step, find where headroom is small and leave that segment alone, test with a guardrail that protects what users value, and recheck the result after launch.

11. Interview questions and model answers

Seven questions: two Beginner, two Intermediate, three Hard. Model answers are **Illustrative**; every figure is an assumption. Each is written against the six behaviors in section 1 and uses the spine in FW-C08-01. The answers are fuller than the first pass of a real round; a spoken version keeps the same order with fewer words. Where a real product is the setting, the answer reasons from user needs and public information, and states no internal fact.

IQ-C08-01 · Beginner · product design · PS · Google

Prompt. How would you improve the inbox of a widely used email app for people who receive more than 100 emails a day?

Rubric.

- **Baseline:** Lists features (smarter search, more labels, a dark mode) without saying who the user is or where the inbox fails them.
- **Strong:** States what the product is for, chooses a segment and a step, names a specific problem, offers different types of move and recommends one.
- **Standout:** Treats a wrong automatic decision as the main risk, protects people who rely on the current view, sizes the gain and sets a guardrail on missed important mail.

Model answer. I'll assume the existing app on phones and the web, and a goal of making the inbox the place where heavy email users get their work done. I'll note that I know the product only from public information and use.

State. Role: a utility that anchors the user to the company's wider account and services. Users: people with high volume (managers, sales, support, executives), moderate users, and casual users. Funnel: mail arrives, the person triages (decides what needs action), reads, acts, then files or deletes. Metrics I'd expect it to follow: daily use, time in the app, and search use. Constraints: years of mail in people's folders and labels, habits around a chronological view, and the same behavior across devices.

Area. I choose people receiving 100 or more emails a day at the triage step. Their reach is smaller than the whole base, perhaps 5% of users (invented), but the pain is highest, the effort repeats every day and they influence which tool their teams use. I give up casual users, who have little to improve.

Problem. A heavy user cannot tell quickly which messages need a reply from them and which only inform. The workarounds are the evidence: starring, moving mail into folders by hand, and rereading the inbox several times a day. I'd verify by watching three people triage and by looking at how many messages people open and then leave unanswered.

The four moves. Fix: make the existing importance signals more accurate and more visible, since wrong ones teach users to ignore them. Add: a "needs reply" section built from signals such as being addressed directly, a question in the text and the person's past replies to that sender. Remove: take out manual steps in triage, such as having to label a thread to keep track of it. Reposition: treat the inbox as a task list.

Size. If 5% of 500,000,000 users (invented) are heavy, that is 25,000,000 people. If triage takes 40 minutes a day and the section saves 10% of it, that is 4 minutes a day per person; with a 30% haircut for people who ignore the section, about 2.8 minutes, so about 70,000,000 minutes a day across the group, or about 1.2 million hours a day. The number is large because of scale, and I'd still check it with a test, since it assumes people trust the section.

Recommendation. I recommend the Add, with the Fix, in the first version: a "needs reply" section above the normal inbox, never replacing it, with the reason shown ("addressed to you, contains a question"). It addresses the triage problem by giving up a more ambitious reposition. Messages are only highlighted, never hidden, because a wrong decision that buries an important email is the worst failure and the one that destroys trust.

Metrics. Primary: the share of messages in the section that the user replies to within 24 hours. Input: the share of heavy users who open the section daily. Guardrail: the share of replied-to messages that the section had not flagged, which measures what it misses, and it should stay under 5% (invented). Counter metric: the share of flagged messages that the user dismisses without action, which measures noise.

Risks. Legacy users who rely on the chronological view keep it as the default, with the section optional. Misjudged importance could favor some senders, so the signals are shown and explained, and the user can turn them off. I'd change my mind if watching users showed that triage was fast already and the cost was in writing replies, in which case the lever is drafting help and not sorting.

Company note. Google is the setting only: Inferred from candidate reports that improvement prompts at Google can concern its own products at large scale [unverified: candidate-reported]. No Google data or product behavior is claimed.

Follow-ups.

- **Q.** What if automatic prioritization hides an important email? **A.** "A missed important message costs far more than extra clutter, so I would design to avoid hiding. All mail stays in the inbox; prioritization only adds a section or a marker, never removes mail from the main view by default. Every automatic decision can be undone in one tap, and moving a message into or out of priority teaches the system. I would measure the missed-important rate by user reports and by messages that users move from the low section to priority, with a stop line set in advance, and release first to an opt-in group before changing the default for anyone."
- **Q.** How would you size the gain and judge whether it is worth it? **A.** "Through time saved for the heavy segment. A person with 100 emails a day who spends about 8 seconds triaging each uses roughly 13 minutes a day; if prioritization cuts that by 20%, it saves about 2.7 minutes a day. If heavy users are 15% of the base, the average gain is about 0.4

minutes per user a day, and I would also look at satisfaction and retention in the heavy group. The measure is time to first action on priority mail and triage time, not opens, and the guardrail is the missed-important rate. If the saving is below a minute for heavy users, the feature is probably not worth the risk."

IQ-C08-02 · Beginner · product design · PS

Prompt. Improve the order-tracking screen of a food-delivery app.

Rubric.

- **Baseline:** "Add a map and push notifications," with no reasoning about the problem.
- **Strong:** Identifies the customer's problem while waiting, chooses a fix over an addition where it is better, sizes it and gives a metric.
- **Standout:** Sees that an honest estimate can hurt conversion at checkout, names the second-order effect and keeps a counter metric on it.

Model answer. The product is a marketplace and its role is revenue and retention: customers who get food when they expect it come back. The segment is customers who have ordered and are waiting. The step is from "order placed" to "delivered," and the pain is uncertainty: they do not know when the food will arrive or why it is late. The evidence I'd look for is contacts to support asking "where is my order?", which I'd assume at 6 per 100 orders (invented), and the cost of each contact, assumed at \$3 (invented).

Moves. Fix: make the estimated time accurate, and show a range with a reason when something changes ("the restaurant is taking longer"). Add: a live map of the courier. Remove: take away the minute-by-minute countdown that shows precision the app does not have; a countdown that is wrong damages trust more than a range. Reposition: none. I recommend the Fix with the Remove: honest, accurate estimates, which cost less than a map and address the cause of the contacts.

Size. With 1,000,000 orders a month (invented), 6% contact support: 60,000. If the change cuts contacts by 25%, that is 15,000 fewer; with a 30% haircut, 10,500, or about \$31,500 a month in support cost. The cost saving is modest, and the larger effect is likely on satisfaction and reordering, which I'd measure and not assume.

Second-order effect. An honest range is longer than an optimistic number, and the estimate shown at checkout affects whether a customer orders at all. If I show "35 to 45 minutes" where the app said "30," conversion could fall. So the checkout estimate and the tracking estimate must come from the same model, and I'd test checkout conversion as a guardrail.

Metrics. Primary: support contacts about delivery time per 100 orders. Input: the share of orders delivered within the shown range. Guardrail: conversion at checkout. Counter metric: the share of customers who cancel after seeing a longer estimate, which shows whether honesty is driving them away. I'd change my mind if the data showed that most contacts were about missing items rather than timing, in which case the lever is order accuracy.

Follow-ups.

- **Q.** An honest delivery estimate can lower conversion at checkout. How do you handle it?
A. "I would test it and compare the costs. Suppose showing a longer, honest estimate reduces checkout conversion by 1%, and a typical order earns \$4 of margin, so the loss is about \$0.04 per checkout attempt. If it cuts where-is-my-order contacts from 6 to 4 per 100 orders, at \$3 each that saves about \$0.06 per order, and it should raise repeat orders because the food arrives when expected. So the decision rests on the 30-day reorder rate and contacts together. I would run the test for four weeks, and ship if net contribution per customer is positive and complaints about lateness fall."
- **Q.** What do you show when an order is late? **A.** "A proactive message with a new estimate and a short reason, before the customer has to ask. Beyond a stated threshold, such as 15 minutes late, offer a choice: keep waiting, cancel with a full refund, or accept a credit. I would measure contacts per 100 orders, refund and credit cost, and how often the estimate changes, because an estimate that keeps shifting damages trust more than one that is wrong once. The guardrail is that the credit rate stays within a budget set with finance."

IQ-C08-03 · Intermediate · product design · PS · Apple

Prompt. Improve a notes app that syncs across phone, tablet and laptop.

Rubric.

- **Baseline:** "Add tags, a better editor and handwriting support," with no reasoning about the cross-device experience.
- **Strong:** Focuses on the cross-device problem, names legacy users and consistency as constraints, and chooses a fix over a feature.
- **Standout:** Treats trust as the product, chooses the metric that matches the failure (lost content, not engagement), and handles encryption, older versions and offline editing.

Model answer. I'll assume the notes app that comes with the user's devices, and I know it only from public information and use. Role: a utility that makes each device more useful and keeps users within the same set of devices; its value grows with every device a person adds, so cross-device behavior is central. Users: heavy note-takers (students and professionals with thousands of notes), casual users (lists and reminders) and people who share notes. Funnel: capture, organize, find, revisit on another device, share. Constraints: years of notes and folder structures, consistent behavior across every device, offline editing, encrypted notes, and devices running older versions.

Area. I choose heavy note-takers at the moment of opening a note on a second device. Their stakes are the highest because their notes are their work, and the step has a specific failure: a note edited on one device appears stale or differs on another, or two edits clash and one is lost. I give up casual users, who rarely feel it. I'd verify by looking at how often a note changes on two devices within a short time and what users do after a conflict.

Problem. "I edited a note on my phone, opened it on my laptop and saw the old version, or lost a paragraph." The workarounds are the evidence: copying text into a message to myself, saving a second copy, or emailing notes to myself. These show a loss of trust, which for a notes app is the heart of the product.

Moves. Fix: show a clear sync state ("up to date" or "syncing") and handle a conflict by keeping both versions and not by overwriting. Add: a version history, so any loss can be undone. Remove: nothing. Reposition: none. I recommend the Fix and a small Add: visible sync state, a keep-both rule for conflicts, and a short version history.

Constraint scan. Legacy users: existing notes and folders must be untouched, and no change to how a note is stored should require action. Ecosystem: the behavior must work the same on every device, so I'd stage the rollout across all of them, and on devices on older versions the new behavior degrades gracefully, with no loss. Privacy: for encrypted notes the conflict is resolved on the devices, and the service never needs to read the note.

Size. Assume 20,000,000 people use two or more devices (invented). If 4% hit a conflict or a stale view in a month, that is 800,000; if 10% of those lose content, 80,000. A keep-both rule that prevents 80% of losses prevents 64,000 losses a month. The count is small against the base, and the severity is high: one lost note can end a person's use of the app. So the size does not say "skip it"; it says to measure the right thing.

Metrics. Primary: lost-content incidents per 100,000 multi-device users per month, from reports and detected overwrites, with a target near zero. Input: the share of conflicts resolved by keeping both versions. Guardrails: sync delay at the 95th percentile, and storage used by duplicate versions. Counter metric: complaints about duplicate notes, since keep-both can clutter a library. I'd change my mind if the data showed that conflicts were rare and the common complaint was finding notes, in which case the area is search.

Company note. Apple is the setting only: Inferred from third-party reports that improvement prompts at Apple concern its own devices and weigh consistency across them [unverified: candidate-reported]. No claim is made about how any existing notes product works.

Follow-ups.

- **Q.** How do you handle a note edited offline on two devices? **A.** "The rule is never lose content. When the same note changed on both devices, the app merges non-overlapping changes automatically, and where changes collide it keeps both versions and shows a conflict copy that the user can reconcile. A version history of about 30 days lets users recover an earlier version. The main metric is lost-content incidents, with a target of zero and every incident investigated; secondary measures are conflicts per 1,000 syncs and the time to reconcile. I would test with deliberate offline edits on several devices before release."
- **Q.** How would encryption and older versions affect your plan? **A.** "End-to-end encryption means the server cannot read notes, so features such as server-side search, merging by content and recovery depend on keys the user holds. I would run search on the device, keep version history encrypted with the same protection, and be clear about recovery: if a user loses their key, the content may be unrecoverable, so I would offer a recovery option and explain the tradeoff plainly. The guardrails are lock-out incidents and the

share of users who cannot find an old note. Older app versions need a migration path so that no note is stranded."

IQ-C08-04 · Intermediate · product design · PS · Apple

Prompt. Improve the walking experience in a maps app.

Rubric.

- **Baseline:** "Add augmented-reality directions and more landmarks."
- **Strong:** Segments walkers, chooses a moment in the walking journey, names the problem, offers different moves and recommends one with a metric.
- **Standout:** Uses the hardware (a watch, haptics) to keep the phone in a pocket, addresses distracted walking and battery, and protects users who rely on audio or who drive.

Model answer. I'll assume the built-in maps app on phones and watches. Role: a utility that makes the devices more useful, with walking as one of several modes alongside driving and transit. Users: commuters walking to transit, visitors in an unfamiliar area, and errand walkers. I'll organize the walking journey as plan, start, move, arrive.

Step	Reach	Pain or loss	Headroom	Choice
Plan the route	High	Low	Low	Rejected
Start walking: which way do I go?	High	Medium to high: the person does not know which direction to walk	High	Chosen
Move: follow the turns	High	Medium	Medium	Second
Arrive: find the right entrance	Medium	Medium	Medium	Rejected for now

Problem. At the start of a walk, the person standing on a street corner cannot tell which way to go. The map dot may not show the direction they face, so they walk a few steps, watch the dot, and correct. The workaround is the evidence (walking the wrong way for half a minute). Visitors in an unfamiliar area feel it most, since they have no sense of direction to lean on.

Moves. Fix: improve the accuracy of the facing direction and give a plain first instruction ("walk toward the park, on Oak Street"). Add: turn cues on the watch, a tap for left and a different tap for right, so the person need not look at the phone, which also helps in the rain and in a crowd. Remove: reduce the map's detail while walking, since clutter makes the route harder to see. Reposition: none. I recommend the Fix with the watch cues. They use what the devices are good at (sensing direction, haptics on the wrist) and they address the start.

Size. Assume 50,000,000 navigation sessions a week (invented), 8% on foot: 4,000,000. If 15% begin in the wrong direction, that is 600,000; the change cuts it by 40%: 240,000; with a 30% haircut, 168,000 avoided wrong starts. At 1.5 minutes saved each, that is 252,000 minutes, or about 4,200 hours a week. The gain is in time and confidence, and I'd say the value is the experience and not a number tied to revenue.

Constraints and risks. Legacy users: drivers and transit riders must see no change. Safety: the point of the haptic cues is to reduce screen-watching while walking, and I'd test that screen-on time falls. Accessibility: people who use audio guidance should get the first instruction spoken as well. Battery: continuous direction sensing costs power, so it runs only while navigating. Privacy: the facing direction is computed on the device.

Metrics. Primary: the share of walking routes whose first 30 seconds follow the route direction, inferred from location. Input: the share of walkers using watch cues. Guardrails: battery use per walking hour, and screen-on time while walking, which should fall. Counter metric: the share of users who turn the cues off, since cues that annoy will be disabled. I'd change my mind if the data showed that wrong starts were rare and the real loss was at the arrival (the wrong entrance), in which case I'd move to that step.

Company note. Apple is the setting only: **Inferred** from third-party reports that improvement prompts at Apple concern its devices and the experience across them [unverified: candidate-reported]. No claim is made about how any existing maps product works.

Follow-ups.

- **Q.** How would you keep people from staring at the phone while they walk? **A.** "By moving guidance off the screen. Haptic taps on a watch for left and right turns, short audio prompts through earbuds, and a simple lock-screen view reduce the need to look. I would measure the share of walking time with the screen on and the number of times a user stops to check, with safety reports as a guardrail. I would make audio guidance work well with screen readers and for people who rely on audio, and keep the visual map for those who want it, since a feature that helps most users and breaks for blind users is not a good design."
- **Q.** How would you protect battery life? **A.** "By treating battery as a requirement with a budget. Use lower-power location updates when the route is straight, and wake the screen only at decisions. I would measure battery drain per 30-minute walk against a baseline, with a target of no more than about 3 percentage points more than a walk without guidance, and test on older devices. If a mode costs more, I would say so in the interface and let the user choose. The guardrail is the rate of users who turn off guidance mid-walk because the battery is low."

IQ-C08-05 · Hard · product design · PS · Meta

Prompt. Many users of a social app say they spend more time on it than they want. How would you improve the product?

Rubric.

- **Baseline:** "Add a screen-time reminder," with no view of the conflict with the business or of who is affected.
- **Strong:** Names the tension between time spent and user wellbeing, chooses a segment, offers different moves, and sets metrics that allow time spent to fall.
- **Standout:** Agrees in advance how much engagement the company will trade, protects younger users with defaults, applies Goodhart's law, and says what would change the plan.

Model answer. I'll start with the tension, since it shapes the answer. I'll assume the product earns revenue from advertising, so time spent matters commercially (an assumption I'd confirm), and that users are telling us the product sometimes works against their own goals. That is a goal conflict, and the interviewer will want to see how I handle it, not how I avoid it.

State. Role: a core product and revenue engine. Users: people who say they spend more than they want, which I'd define by a short survey and by behavior (late-night sessions, long unplanned sessions), not only by volume; people who spend a lot and are content; and people who use it briefly. Funnel: open, scroll, interact, close. A feed that does not end has no natural stopping point.

Area. I choose adult users who report that they spend more time than they want, at the moment of a long unplanned session. Their stated regret is a direct signal, their number is large, and the product's design contributes to the problem. I treat younger users separately with stricter defaults, because they are more vulnerable, and I'd involve safety and legal teams from the start; rules differ by country [V: rules differ by country, for example India's DPDP Act treats anyone under 18 as a child].

Problem. The feed has no natural end. People do not notice time passing, and the product offers no moment to decide to stop. Workarounds: deleting the app and reinstalling it, using the phone's own limits, and logging out.

Moves. Fix: make the existing time tools easier to find and use. Add: a "you're all caught up" point after new content ends, and an optional reminder after a session length the person chooses. Remove: the next-item autoplay after the caught-up point. Reposition: from maximizing time to maximizing value, a change in what the product optimizes. Public accounts of one such shift exist: in early 2018 Facebook's chief executive said a change to the News Feed would reduce time spent in favor of more meaningful interactions, and the company reported a drop in time spent Public [V: CS-C05-02, Facebook's January 2018 post and call] (C05, CS-C05-02). I recommend the Add and the Fix first, with the Reposition as the longer question.

Metrics, and the trade. Primary: the share of surveyed users in the segment who say they are satisfied with how they spend time on the product. That is a self-report and a proxy, and I'd pair it with behavior. Guardrails: long-run retention (a product that is healthier to use should keep people over years) and the reach of creators, who depend on attention. Counter metric: time spent and sessions per user, which I expect to fall, and I'd agree before launch how much of a fall is acceptable (say no more than 3% in the first quarter, invented), so that the team and leadership decide the trade in advance and not under pressure. Goodhart's law applies: if satisfaction became a target, it could be raised by changing how the survey is asked, so I'd keep the question fixed and audit it.

Risks. Winners and losers: users in the segment gain; creators who depend on long sessions lose reach; advertisers see fewer impressions. Defaults have a large effect on what people end up with (MM-defaults, C07), so for younger users the reminders and limits are on by default. I'd reverse the break point if retention among the segment fell without a gain in reported satisfaction. I'd change my plan if the survey showed that the regret came mostly from one feature, and a targeted change would do more than a general break.

Company note. Meta is the setting only: **Inferred** from candidate reports that improvement prompts at Meta can pair engagement questions with wellbeing and integrity tradeoffs [unverified: candidate-reported]. No Meta data or internal decision is claimed.

Follow-ups.

- **Q.** How much engagement are you willing to trade, and who decides? **A.** "This must be decided before the work, with leadership, in writing. For example, accept up to a 2% fall in daily time among users who opt into limits if satisfaction among them rises by a stated amount, and set protective defaults for younger users even at a larger cost. The decision record names the trade, the measure and the date it will be reviewed. Without it, teams drift back to the number the company rewards. Goodhart's law applies in both directions: if we target a well-being score, it can be gamed, so I would keep several measures."
- **Q.** How would you tell whether the changes help users and not just reduce time? **A.** "I would look at what the lost time was and what users say. Compare time well spent from a short survey item, 90-day retention, and active interactions for users who got the changes against a holdout. If time drops and satisfaction rises with retention flat, the change helped. If time drops and satisfaction does not move, I only reduced usage. I would also check the users who opted in, since they differ from others, and avoid reading the effect from them alone."

IQ-C08-06 · Hard · product design · PS

Prompt. An enterprise software product has a legacy report feature used by 4% of customers that consumes 20% of the team's maintenance effort. The team wants to remove it. What do you do?

Rubric.

- **Baseline:** "Remove it, since few customers use it," or "keep it, since customers pay for it."
- **Strong:** Asks who the 4% are and what they pay, applies the fence test, compares options and sets a migration path.
- **Standout:** Quantifies revenue at risk with and without a path, chooses a path that gives customers time and a replacement, defines how success and harm will be measured, and says when to stop.

Model answer. I'd treat this as a removal decision with legacy users, and the number 4% is the start of the questions, not the answer. First the fence test (FW-C08-05): who is in the 4%, what do they pay, why do they use it, and why was it built? A small share of customers can be a large share of revenue, and enterprise customers often build routines around a report.

Assumptions. 5,000 customers (invented), so 200 use the feature. They average \$30,000 a year (invented), so \$6,000,000 a year is at stake. The 20% of maintenance effort equals two engineer-years on a team of ten (invented), at \$200,000 each fully loaded, so \$400,000 a year.

Options.

Option	Revenue at risk	Cost and effect
A. Remove it now	15% churn among the 200 (invented): \$900,000	Frees \$400,000 a year; breaks routines; damages trust
B. Keep it and freeze it	Near zero	Saves part of the effort; the debt stays
C. Replace it with a better report and migrate users over 12 months	3% churn (invented): \$180,000	Frees \$400,000 a year after the transition; costs migration work
D. Keep it as it is	Zero	Keeps all the cost and the opportunity cost of two engineers

The arithmetic: 15% of \$6,000,000 is \$900,000, more than the \$400,000 saved, so an abrupt removal loses money in the first year. With a path, 3% of \$6,000,000 is \$180,000, less than the saving, and the saving then continues.

Recommendation. I recommend C: build the replacement, announce the change early, migrate the data and reports automatically, keep the old feature running for 12 months, and then remove it. It gives up the immediate saving. First I'd talk to a sample of the 200, especially the largest, to learn why they use it, and the replacement should meet that need and not just the feature's description. I'd check contracts and any commitment that names the feature.

Metrics. Primary: the share of the 200 accounts migrated by the deadline. Guardrails: churn and contraction among the 200 compared with other accounts, and support tickets. Success: maintenance effort actually freed, and what it is used for; if it goes to the roadmap, I'd say what shipped. Counter metric: sentiment among affected accounts.

Risks and stopping rule. The loudest accounts may be the strategic ones, so I'd treat each large account individually. I'd stop the removal, or extend the date, if migration is below a threshold set in advance (say 70% by month nine, invented) or if a strategic account says it will leave. I'd change the plan entirely if the interviews showed the feature does something nothing else does, in which case the product is missing a capability, and the question becomes how to build it well.

Follow-ups.

- **Q.** What do you do for the customers who rely on the report? **A.** "Give them a path and time. Build a replacement that covers most of what the report does (say 80% of the use cases), provide an export for the rest, announce the removal 12 months ahead, and give the largest accounts a named contact and help to migrate. For the few customers whose needs cannot be met, offer a paid support arrangement for a limited time. I would track migration progress by account, and tell customers early, with dates, what will change and what will not."
- **Q.** How would you decide when to stop the removal? **A.** "With triggers set before the announcement. For example, pause if more than 3 of the 200 affected accounts cancel because of the change, if support tickets about the report exceed a set level, or if accounts representing more than 25% of the affected revenue have not migrated by month nine. When a trigger is hit, extend the date and fix the cause: a missing capability,

a migration tool, a pricing issue. The aim is to reduce maintenance cost without losing the revenue that the removal was supposed to protect."

IQ-C08-07 · Hard · product design · PS

Prompt. A mature productivity app has had flat weekly active users for six quarters, although the team has shipped many winning experiments. What would you do?

Rubric.

- **Baseline:** "Run more experiments and add more features."
- **Strong:** Questions why wins do not add up, looks at cohorts and segments, and considers larger moves.
- **Standout:** Reconciles the sum of experiment lifts with the actual outcome, names the local-maximum risk, tests a new segment or job with a cheap bet, and sets expectations for the first quarter.

Model answer. I'd start with the puzzle in the prompt: many winning experiments and a flat result. They cannot both be fully true, so something is wrong in how the wins are counted, or the wins are being offset.

Reconcile first. Suppose the team's experiments claim an added 9% of weekly actives over six quarters (invented), and the actual change is zero. I'd list the usual reasons. Novelty: lifts fade after a few weeks, and a test measured over two weeks overstates them. Overlap: two wins can improve the same users and not add. Shifts: a win in one place can reduce use elsewhere (cannibalization within the product).

Proxies: experiments may be won on a metric (clicks on a feature) that is a [[proxy-metric]] for value and is not the number that matters. Offsets: other things are pulling the number down, such as older cohorts decaying faster or new-user cohorts getting smaller. I'd check each with data, using cohorts (C05): are older cohorts retaining as before, and are newer cohorts arriving at the same rate and activating as well? The experiment mechanics are C14.

What the answer is likely to be. If the flat number is a balance between decay and acquisition, the improvements are keeping the line level and the real question is growth. If new cohorts are healthy and older ones decaying, the problem is long-term retention. If new cohorts are small, the product may be saturating its market. Each leads to a different move.

Moves. Fix: remove what is hurting retention for older cohorts (for example, complexity added over six quarters). Add: a capability for a job the product does not serve today. Remove: features that few use and that make the product harder to learn. Reposition: a different segment or job. The local-maximum risk is real: a team that optimizes steps one at a time will not find a better position, so alongside the steady work I'd run one or two larger, cheap bets, such as interviews and a prototype for an adjacent segment, with a stated criterion for success, and I'd protect them from the weekly metric.

Expectations. I'd tell leadership that the first quarter should produce a diagnosis and a small number of bets, not growth, and I'd agree what we would look at: the reconciled lift, cohort curves, and the results of the bets. The primary metric stays weekly actives for the core

segment, with retention by cohort as an input, and a guardrail on the health of the core experience (task completion, support load). A counter metric is complexity: the number of features a new user must learn.

What would change my mind. If reconciliation showed that the experiment wins were real and the offset was a one-off event such as a change in how users are counted, the answer would be a data fix and not a product move. If the larger bets failed on their criterion, I'd concentrate the team on retention for older cohorts.

Follow-ups.

- **Q.** How do you reconcile many winning experiments with flat weekly actives? **A.** "I would find where the wins go. Common reasons are novelty (lifts fade after the test window), overlap (two wins that move the same users and do not add), measuring a metric among engaged users who were already active, and shifts in the definition of an active user over six quarters. The most reliable check is a cumulative holdout: keep 1% of users on the product as it was a year ago and compare their weekly activity with everyone else's. If the gap is much smaller than the sum of the lifts, the claimed wins were overstated. I would also check seasonality and acquisition changes."
- **Q.** What bet would you make outside the local maximum? **A.** "A small test of a new job or segment, for example shared workspaces for teams of three to five. Offer it to a few thousand users, measure invites sent, invitees who become active, and retention of invited users after four weeks, with cost of the team as small and time boxed to a quarter. I would set the expectation with leadership that the first quarter is for learning and may not move weekly actives, and agree the kill criteria in advance. A bet like this tests whether there is growth outside the current use."

12. Exercises

Ten exercises. Each has a model solution and an alternative. Figures are assumptions unless marked.

EX-C08-01 · What is this product for?

Problem. Assign a role (revenue engine, growth engine, retention anchor, strategic bet, utility) to each of these, and say what "an improvement" would mean for each: (a) the built-in calculator on a phone; (b) a free email service from a company that sells advertising; (c) a paid project-management tool sold to companies; (d) a video app launched to hold a position against a rival; (e) a free app that gets people into a company's paid app.

Context. You are starting an improvement answer and must state the role.

Data. None beyond the descriptions.

Constraints. Two sentences each.

Expected approach. Use the role line of the state card (FW-C08-02), and state the role as an assumption.

Solution. (a) Utility: an improvement is accuracy, speed and rarely-surprising behavior, even if time spent falls. (b) Retention anchor or growth engine, with advertising as revenue: improve what keeps people returning and brings their other activity into the account, and watch that changes do not cost trust. (c) Revenue engine: improve what drives renewals and expansion, for the buyer as well as the daily user. (d) Strategic bet: an improvement is progress on the position (reach, quality versus the rival), accepting lower short-term returns. (e) Growth engine: improve the conversion of free users to the paid app, and watch for a change that raises sign-ups and lowers quality.

Alternative solution. Some roles overlap; choose the primary one and name the second, as in (b).

Tradeoffs. Naming a role commits you to a view of what matters; ask the interviewer to correct it, since a wrong role changes the whole answer.

EX-C08-02 · Headroom, then downstream value

Problem. A shopping app's funnel for a month: 100,000 people view a product page; 60,000 add to cart; 30,000 start checkout; 24,000 pay. Benchmark rates for comparable apps: add to cart 70%, checkout start 60%, payment 95%. Which step should you improve to add the most paid orders?

Context. The goal is more paid orders. You have one quarter.

Data. Current rates: view to add 60%; add to checkout start 50%; checkout start to pay 80%.

Constraints. Assume newly won users convert at the current downstream rates.

Expected approach. Compute headroom in users at each step, then follow those users downstream to paid orders (FW-C08-03, FW-C08-06).

Solution. Step 1 (view to add): $\text{gap} = (70\% - 60\%) \times 100,000 = 10,000$ users; downstream conversion is $50\% \times 80\% = 40\%$: 4,000 paid orders. Step 2 (add to checkout start): $\text{gap} = (60\% - 50\%) \times 60,000 = 6,000$ users; downstream 80%: 4,800 paid orders. Step 3 (checkout start to pay): $\text{gap} = (95\% - 80\%) \times 30,000 = 4,500$ users; downstream 100%: 4,500 paid orders. The checkout start has the most downstream value, 4,800, though the first step has the largest gap in users. Apply a haircut, since marginal users convert less than average, and check the pain at step 2 with data before choosing.

Alternative solution. Choose step 1 if it is far cheaper to improve, and say that the difference between 4,000 and 4,800 is within the uncertainty of the assumptions.

Tradeoffs. Benchmarks are rough and the downstream-rate assumption is generous; the point is that the largest raw gap is not automatically the best step.

EX-C08-03 · Fill in the selector

Problem. A language-learning app wants more people to complete their first week. Rate these candidate areas on reach, pain or loss, headroom and fit, and choose one: (a) new users at sign-up; (b) new users at the first lesson; (c) users on days 3 to 5, when streaks break; (d) users choosing a plan.

Context. You have no data beyond what is below.

Data. Invented figures. Sign-up: 90% of arrivals complete it. First lesson: 70% of sign-ups start it. Days 3 to 5: 40% of those who started a lesson miss a day and 55% of those do not return. Plan choice: 12% choose a paid plan, which is typical.

Constraints. Seven minutes.

Expected approach. Use the selector table (FW-C08-03) and name the rejected rows.

Solution. (a) Reach high, loss low (90% complete), headroom low: reject. (b) Reach high, loss medium (30% do not start), headroom medium; fit medium. (c) Reach medium (people who started), loss high (of the 40% who miss a day, 55% leave, so 22% of starters), headroom high; fit high for "complete the first week." (d) Reach low, loss not relevant to the goal: reject. Choose (c), the break in the first week. Name (b) as second, and say that the first lesson step is worth a check because a person who never starts never reaches days 3 to 5.

Alternative solution. Choose (b) first if the data show that most of the lost people never started a lesson, since the largest absolute loss is there: 30% of sign-ups at (b) against 22% of starters at (c), and the base at (b) is larger.

Tradeoffs. (c) is the better fit for the goal and (b) may have more users to win; the two differ by what is known, so say which assumption would flip the choice.

EX-C08-04 · Size it

Problem. A recipe app has 500,000 weekly users. 20% reach a "create a shopping list" step, where 50% complete it today. A fix is expected to raise completion to 55%. Each completed list is worth \$2 in revenue (invented). Use a 40% haircut. What is the weekly and annual value, and what is the payback if the change costs \$150,000 in team time?

Context. The team has two candidate projects and needs to compare them.

Data. As above.

Constraints. Show each step.

Expected approach. Use the sizing ladder (FW-C08-06).

Solution. Reach: $500,000 \times 0.20 = 100,000$. At 50%: 50,000 complete. At 55%: 55,000, so 5,000 more. Haircut 40%: $5,000 \times 0.6 = 3,000$. Value: $3,000 \times \$2 = \$6,000$ a week; $\times 52 = \$312,000$ a year. Payback: $\$150,000 \div \$6,000 = 25$ weeks. Check by a second route: a five-point lift on a base of 100,000 is 5,000, and a 40% haircut leaves 3,000, the same. Say the range: if the haircut were 60%, the weekly value would be \$4,000 and the payback 37.5 weeks.

Alternative solution. If the \$2 value comes from a correlation (people who complete lists spend more), credit part of it: at half, \$3,000 a week and a payback of 50 weeks.

Tradeoffs. A longer payback is not a reason to decline automatically; compare it with the other project's payback and with strategic value.

EX-C08-05 · Four moves

Problem. Problem: in a photo-printing app, customers abandon the "choose a size" step because there are 14 sizes and the differences are unclear. List a fix, an add, a remove and a reposition, and recommend one.

Context. The app is mature and the step has high traffic.

Data. 38% of customers who reach the step leave (invented).

Constraints. One line per move.

Expected approach. Use FW-C08-05, and test any removal with the fence questions.

Solution. Fix: label each size with what it is good for ("fits a standard frame") and preselect the most common. Add: a size recommender that asks where the print will go. Remove: cut the 14 sizes to six, keeping those that account for most orders (check that some rare sizes are not needed by a segment such as professional photographers). Reposition: sell "wall displays" and not "prints," with a few curated sets. Recommend the Fix with a trial of the Remove for the least-ordered sizes, because they are cheap, address the cause (too many unclear options) and are easy to reverse.

Alternative solution. Recommend the Remove alone if the order data show that eight of the 14 sizes are under 3% of orders each (invented).

Tradeoffs. Removing choices simplifies the step and may lose customers who need a rare size; keep a "more sizes" route for them.

EX-C08-06 · Fence test

Problem. A team proposes removing: (a) a rarely used keyboard shortcut; (b) a confirmation prompt before deleting an account; (c) an export-to-fax option in a business tool. For each, write the questions you would ask before removing it, and your recommendation.

Context. The product is mature and the team wants to simplify.

Data. (a) used by 0.5% of sessions; (b) skipped by 98% of users, who click through; (c) used by 0.2% of customers, mostly large.

Constraints. Two to three questions each.

Expected approach. Apply the fence test: who uses it, why was it built, what happens to them, how would you know it was a mistake.

Solution. (a) Who uses it, and are they heavy users who value speed? Low risk; remove or move it behind a setting, and measure the heavy users' task time. (b) Why was it built? It guards against an irreversible mistake; 98% click-through shows it annoys most users and protects a few. Do not remove it; make it smarter (require it only for accounts with data, and allow a short undo period). (c) Who are the 0.2%, and what do they pay? Large customers in sectors that still use fax may be high-value and slow to change. Keep it or offer a replacement and a long notice.

Alternative solution. For (b), keep the prompt and test a lighter version, measuring accidental deletions.

Tradeoffs. Every prompt costs the many to protect the few; the right answer depends on how bad the mistake is and whether it can be undone.

EX-C08-07 · Winners and losers

Problem. A note-taking app plans to move its "new note" button from the bottom right to the top left to match a design change on another platform. Fill in a winners-and-losers table.

Context. Heavy users have used the current position for years.

Data. 70% of new notes come from 10% of users (invented).

Constraints. Four rows.

Expected approach. Use FW-C08-07 and name the measure for each row.

Solution. Rows: (1) heavy users, who lose a habit; reaction: mis-taps and irritation; mitigation: a setting to keep the old position for six months and a short notice; measure: time to create a note and the rate of accidental taps. (2) New users, who gain consistency with other apps they use; measure: first-note completion. (3) Left-handed users, who may gain or lose; measure: task time by handedness setting. (4) Users on other devices, who lose consistency unless the change is made everywhere; mitigation: ship on all devices together. Because 70% of notes come from 10% of users, the average could look neutral while the main users are worse off, so measure heavy users separately.

Alternative solution. Do not make the change, and say that consistency with another platform is a weak reason against the habit of the people who create most of the content.

Tradeoffs. A choice that is neutral on average and harmful to the core is a decision to make knowingly, not by default.

EX-C08-08 · Gross versus net

Problem. A subscription app has 100,000 subscribers at \$10 a month. It adds a \$5 plan with ads. After launch, 40,000 people are on the \$5 plan: 15,000 downgraded from the \$10 plan and 25,000 are new. What are the monthly revenues before and after, how large is the net gain, and how much does the gross figure overstate it? At what share of existing subscribers choosing the new plan does the net gain become zero?

Context. The team plans to report the new plan's revenue as growth.

Data. As above.

Constraints. Show the arithmetic.

Expected approach. Count net, not gross (FW-C08-04, cannibalization).

Solution. Before: $100,000 \times \$10 = \$1,000,000$. After: $85,000 \times \$10 = \$850,000$, plus $40,000 \times \$5 = \$200,000$, total $\$1,050,000$. Net gain $\$50,000$ a month. Gross revenue from the new plan is $\$200,000$, four times the net gain. Break-even: if d of the 40,000 come from existing subscribers, the net is $\$5 \times (40,000 - d) - \$5 \times d = \$5 \times (40,000 - 2d)$, which is

zero at $d = 20,000$, so at 50% cannibalization. At 37.5% (15,000 of 40,000) the plan is still positive. Add the ad revenue from the \$5 users and the possibility that some later upgrade, which would improve the picture.

Alternative solution. Compare with a holdout: offer the plan to a random half of new visitors only, so that downgrades from existing users can be seen separately.

Tradeoffs. A cheaper plan can widen reach and reduce revenue per user; the net, not the gross, is the number to report.

EX-C08-09 · A timed full run

Problem. Give a 25-minute answer to: "How would you improve a public-transport app for people who ride every day?"

Context. Self-scored; record yourself. Assume a city, a platform and a goal, and say them aloud.

Data. None. Invent only what you label.

Constraints. Use the short-version budget in section 4: one segment, one step, two moves, a one-line size, one metric, one loser.

Expected approach. Use the full spine (FW-C08-01) and score yourself against the six behaviors in section 1.

Solution. An outline of a strong answer. Clarify and role: a utility and retention anchor for a wider mobility offering; the goal is that regular riders plan and travel with the app. State: users are daily commuters, occasional riders and visitors; funnel is plan, travel, disruption, repeat. Area: daily commuters at disruptions: reach high, pain high when a delay happens, headroom high. Problem: they learn about a disruption late.

Moves: Fix (accuracy and timing of delay alerts); Add (a "leave by" alert that updates). Size: commuters $\times$ trips with delay $\times$ share who act. Metric: on-time arrival among alert users against a holdout. Loser: users who dislike alerts, or who rely on the timetable view; keep it. Reverse if the false-alarm rate is above a threshold.

Alternative solution. Choose visitors at the first trip (how to pay and where to board) and treat commuters as served.

Tradeoffs. The commuter choice has more repeat value; the visitor choice may have more headroom per person; state the criterion.

EX-C08-10 · Handling interventions

Problem. Write a one- to two-sentence reply to each interviewer intervention: (a) "What is this product for?" (b) "Why that part of the product?" (c) "How big could this be?" (d) "What about the people who like it as it is?" (e) "Wouldn't this just move usage from somewhere else?"

Context. You are partway through an answer about improving a video-calling app.

Data. Use any assumptions you state.

Constraints. Two minutes.

Expected approach. Use the table of interventions in section 4: each points to a missing step.

Solution. (a) "For users, it is the easiest way to talk face to face with people who are far away; for the company, I assume it is a retention anchor for the wider account. Tell me if that is off." (b) "Call setup is where most failed calls start; it reaches nearly every user, and I'd expect it to have more headroom than the in-call controls." (c) "If 10% of calls fail at setup and I halve that, with a 30% haircut, I'd expect about 3.5% more successful calls, which is large at this scale." (d) "Regular users have habits around the current setup. I'd keep the old path for a period and measure their completion time separately." (e) "A little: some calls would have succeeded on a second try. I'd count net successful calls against a holdout, not the gross number."

Alternative solution. Record yourself, and check that each reply takes under 20 seconds and returns to the spine.

Tradeoffs. Short replies keep momentum and can skip nuance; offer more if asked.

13. Practice protocol

Timed drills (three a week for four weeks).

- *Week 1: role and state.* Take a product you use (IQ-C08-01, IQ-C08-02, EX-C08-01) and write its five-line state card in five minutes. Say the role, the users, the funnel, the metrics and the constraint aloud.
- *Week 2: area and size.* Use IQ-C08-03, IQ-C08-04, EX-C08-02 or EX-C08-03. Build the selector table and choose with a criterion; then size the move with a chain and a range (EX-C08-04). Twelve minutes.
- *Week 3: moves and losers.* Use IQ-C08-05, IQ-C08-06, EX-C08-05, EX-C08-06 or EX-C08-07. Offer a fix, an add, a remove and a reposition, test any removal, and fill in the winners-and-losers table. Fifteen minutes.
- *Week 4: full rounds.* Run two 45-minute answers and one 25-minute answer (EX-C08-09) on products you have not prepared, with a partner who uses the interventions in section 4.

With a partner. Ask your partner to interrupt with "What is this product for?" and "What about the people who like it as it is?" at random points, and to ask once "Wouldn't this just move usage from somewhere else?" Practice answering in under a minute.

Record and review. Record every full run. Listen once for structure and once for wording. Count how many moves were additions, whether you named a loser, and whether you gave a range for the size.

Feedback checklist (yes, partly, no).

1. Did I state the product's role and the users before proposing anything?

2. Did I choose a segment and a step by reach, pain and headroom, and say what I gave up?
3. Did I describe a specific problem with evidence or a way to get it?
4. Did I offer more than one type of move, including a fix or a removal?
5. Did I size the gain with a visible chain, a haircut and a range?
6. Did I name who loses and how I would protect them?
7. Did I check cannibalization, other devices and dependence on current behavior?
8. Did the metric have a guardrail and a counter metric?
9. Did I say what would make me reverse or change the plan?
10. Did I finish with the two-sentence close from section 4?

Answer journal. For each drill record the date, the product, the role, the area, the move chosen, your size and range, your score out of ten and the step you rushed. After eight drills, look for the step you skip most (usually the losers or the size) and drill that step alone for a week.

14. Implementation playbook

Apply the chapter on real work over the next three weeks. Each deliverable doubles as an interview story.

Days 1 to 3. Write the state card (FW-C08-02) for the product or area you work on: role, users, funnel, metrics, constraints. Check each line against data you can see and mark which lines are assumptions. Share it with your manager and ask what is wrong.

Week 1. Build the selector (FW-C08-03) for your area: three segments and the steps for each, with reach, loss, headroom and fit from real data. Choose where the next effort would be most valuable and compare with what the team is working on. Note any gap and discuss it.

Week 1. For one planned change, fill in the constraint scan (FW-C08-04) and the winners-and-losers table (FW-C08-07). Ask an engineer who knows the history what depends on the current behavior and ask support what customers say about it.

Week 2. Size one proposed change with the ladder (FW-C08-06): write each assumption, apply a haircut, give a range, and check by a second route. Compare with the actual result of a similar past change.

Week 3. Find one feature in your product that few people use and many find confusing. Apply the fence test: who uses it, why was it built, what happens if it goes. Write a recommendation (keep, fix, replace, remove with a path) and present it.

Deliverables that become stories: the state card and selector (a time you chose where to focus using data), the winners-and-losers table (a time you protected existing users when you changed something), and the sizing (a time you estimated impact and were honest about the range). Keep the numbers real, and keep what you did separate from what the team did.

15. Key takeaways and further reading

Key takeaways

1. An existing product has a role, users, a funnel and a history; state them before you propose a change, and treat what you do not know as assumptions to check.
2. Choose the area by reach, pain and headroom, and follow the gain downstream to the outcome you care about; the biggest raw gap is not always the best step.
3. Offer different types of move (fix, add, remove, reposition), and test any removal by asking who uses it and why it exists.
4. Size the gain with a visible chain, haircut it, give a range, and credit only the causal part of any correlation.
5. Name who loses, count net and not gross, and set a guardrail, a counter metric and a condition for reversing.

Further reading

- Marty Cagan, *Inspired* (2nd edition, 2017), and *Empowered* (2020). How strong product teams choose problems and improve live products.
- Melissa Perri, *Escaping the Build Trap* (2018). Why teams add features instead of improving outcomes.
- Alistair Croll and Benjamin Yoskovitz, *Lean Analytics* (2013). Choosing the metric that matters at each stage of a product [V: O'Reilly, 2013, ISBN 9781449335670].
- Sean Ellis and Morgan Brown, *Hacking Growth* (2017). A practical view of improving funnels through experiments.
- Steve Krug, *Don't Make Me Think* (revised edition, 2014). Short and practical on removing friction from interfaces.
- G. K. Chesterton, *The Thing* (1929), in the essay "The Drift from Domesticity", for the passage on the fence that gives the fence test its name [V: Chesterton, *The Thing* (1929), essay *The Drift from Domesticity*].
- Daniel Kahneman and Amos Tversky, "Prospect Theory: An Analysis of Decision under Risk," *Econometrica* (1979), the source of the research on loss aversion [V: *Econometrica* 47(2), 263 to 291, March 1979 (RePEc)].
- Twitter, "Giving you more characters to express yourself" (26 September 2017) and "Tweeting Made Easier" (7 November 2017), now hosted at blog.x.com/en_us/topics/product/2017/. The primary sources for CS-C08-02.